

Methods

1. Logistic Regression -

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

- a. Parameters - C: [0.001, 0.01, 0.1, 1, 10, 100]; solver: ['lbfgs', 'liblinear']

2. Decision Tree with PCA - <https://scikit-learn.org/stable/modules/tree.html>

- a. Parameters - pca__n_components: [5, 10, 15, 20]; max_depth: [None, 1, 5, 10, 20]; min_samples_split: [2, 5, 10]; min_samples_leaf: [1, 5, 10]

3. Random Forest - <https://scikit-learn.org/stable/modules/ensemble.html>

- a. Parameters - n_estimators: [200, 500]; max_depth: [None, 3, 5, 7]; min_samples_leaf: [5, 10, 15]

4. Gradient Boost with PCA - <https://scikit-learn.org/stable/modules/ensemble.html>

- a. Parameters - n_estimators: [200, 400]; learning_rate: [0.01, 0.1, 1]; max_depth: [5, 10, 15]; subsample: [0.8, 1.0]

5. Linear SVM -

<https://scikit-learn.org/stable/modules/generated/sklearn.svm.LinearSVC.html>

- a. Parameters - C: [1e-4, 1e-3, 1e-2, 1e-1, 1, 10, 100]; loss: ["squared_hinge"]; dual: [False]

6. RBF SVM with PCA - <https://scikit-learn.org/stable/api/sklearn.svm.html>

- a. Parameters - n_components: [5, 10, 15, 20]; C: [0.1, 1, 10, 100]; gamma: [0.001, 0.01, 0.1]

7. KNN -

<https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html>

- a. Parameters - n_neighbors: list(range(1, 51, 2)); weights: ["uniform", "distance"]; metric: ["euclidean", "manhattan"]

8. Adaptive Boost with PCA -

<https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.AdaBoostClassifier.html>

- a. Parameters - n_estimators: [200, 400, 500]; learning_rate: [0.01, 0.1, 1]

Results

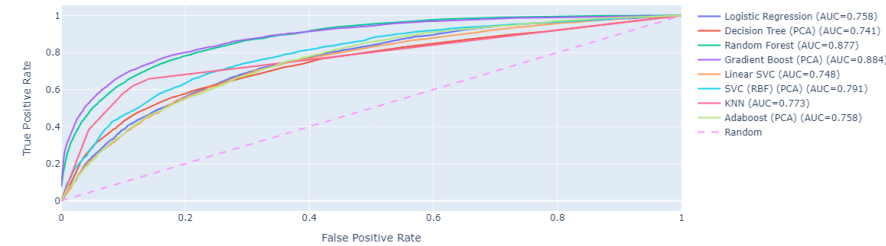
Performance Metrics	Logistic Regression	Decision Tree (PCA)	Random Forest	Gradient Boost (PCA)	Linear SVC	SVC (RBF) (PCA)	KNN	Adaboost (PCA)
balanced_accuracy	0.436	0.480	0.605	0.598	0.408	0.498	0.635	0.319
accuracy	0.441	0.465	0.631	0.667	0.468	0.535	0.659	0.450
micro_recall	0.441	0.465	0.631	0.667	0.468	0.535	0.659	0.450
macro_recall	0.436	0.480	0.605	0.598	0.408	0.498	0.635	0.319
micro_precision	0.441	0.465	0.631	0.667	0.468	0.535	0.659	0.450
macro_precision	0.426	0.459	0.607	0.692	0.439	0.499	0.642	0.276
micro_f1	0.441	0.465	0.631	0.667	0.468	0.535	0.659	0.450
macro_f1	0.410	0.447	0.602	0.622	0.396	0.493	0.637	0.260
macro_auc	0.758	0.748	0.877	0.884	0.748	0.791	0.773	0.747

micro_auc	0.755	0.765	0.889	0.898	0.758	0.802	0.787	0.779
-----------	-------	-------	-------	-------	-------	-------	-------	-------

Confusion Matrix – Gradient Boost (PCA)

True Label	True S	81	9	88	25	682
	True R	33	3	19	144	127
	True P	116	10	314	14	175
	True H	103	124	28	5	40
	True D	712	12	28	17	53
		Pred D	Pred H	Pred P	Pred R	Pred S

Macro-average One-vs-Rest ROC Curves



Confusion Matrix – Random Forest

True Label	True S	109	32	125	85	534
	True R	32	6	29	189	70
	True P	94	38	322	49	126
	True H	80	153	25	14	28
	True D	672	30	35	45	40
		Pred D	Pred H	Pred P	Pred R	Pred S

Confusion Matrix – KNN

True Label	True S	73	32	128	61	591
	True R	46	12	23	184	61
	True P	115	36	339	23	116
	True H	70	182	19	14	15
	True D	657	36	62	19	48
		Pred D	Pred H	Pred P	Pred R	Pred S

Confusion Matrix – SVC (RBF) (PCA)

True Label	Predicted Label					
	Pred D	Pred H	Pred P	Pred R	Pred S	
	True S	142	78	149	64	452
	True R	44	18	21	144	99
	True P	125	72	264	44	124
	True H	116	112	36	19	17
	True D	614	104	35	41	28

Micro-average One-vs-Rest ROC Curves



Conclusion

1. The most accurate method is Gradient Boost, as seen in its highest accuracy (0.667), micro-F1 (0.667), macro-AUC (0.884) and micro-AUC (0.898) in the results table as well as a strong confusion matrix performance, particularly for Class S (682 correct) and Class P (314 correct). Gradient Boost consistently outperforms all other models across both threshold-dependent and threshold-free metrics, indicating that it captures the nonlinear structure of the gesture data more effectively than competing methods. Its strong diagonal dominance in the confusion matrix further demonstrates its robustness across classes, making it the most reliable classifier for this dataset and the recommended method for deployment.
2. Non-linear methods perform the best on this dataset, as seen in the superior accuracy of Gradient Boost (0.667), Random Forest (0.631), SVC-RBF (0.535), and KNN (0.659) compared to linear models such as Logistic Regression (0.441) and Linear SVC (0.468). This is also seen in the ROC curves where Gradient Boost (AUC=0.884), Random Forest (AUC=0.877), and SVC-RBF (AUC=0.791) outperform linear models, and in confusion matrices where these models show stronger diagonal accuracy. This pattern aligns with theoretical expectations: gesture-phase sensor data contains strong nonlinear interactions between accelerometer and gyroscope features, which linear decision boundaries cannot adequately represent. The ROC curves and confusion matrices reinforce this, showing that non-linear models achieve better class separation and fewer misclassifications across all classes.
3. Class imbalance strongly affects model behaviour and influences classification accuracy, as seen in the confusion matrices with the models performing the best on Class D (the majority class) and even strong models misclassifying minority classes more often like classes H and R. The dominance of Class D leads models to favour this class, inflating micro-metrics while masking poorer performance on minority classes. This behaviour is typical in imbalanced multi-class problems, where models tend to optimise for the majority class unless explicitly corrected. The confusion matrices clearly show reduced sensitivity for minority classes, suggesting that future work should incorporate imbalance-handling techniques such as resampling, class-weighted losses, or focal loss.

References

Me: historically, what classifying method has performed the best on this dataset =
`fetch_openml(data_id=4538, as_frame=False)`

AI: `fetch_openml(data_id=4538)` corresponds to the `GesturePhaseSegmentationProcessed` dataset.

Properties of this dataset:

- Multiclass classification
- Time series features extracted from motion sensors
- Strong feature interactions
- Moderate dimensionality
- Nonlinear decision boundaries

Because of these properties, tree ensemble methods historically perform the best on this dataset.

Historically strongest performing model families

1. Gradient Boosted Trees (best overall)

Examples:

- XGBoost
- LightGBM
- GradientBoostingClassifier

Reasons:

- Captures nonlinear feature interactions
- Handles heterogeneous feature distributions
- Boosting sequentially corrects errors from previous trees

Typical benchmark outcome: highest ROC AUC and balanced accuracy.

2. Random Forest

Reasons:

- Ensemble averaging reduces variance
- Works well with complex feature interactions
- Minimal preprocessing required

Performance:

- Usually slightly below gradient boosting
- Very stable across random splits.

3. Decision Trees

Reasons: Dataset structure suits axis aligned splits.

Limitations:

- High variance

- Single tree underfits relative to ensembles.

Performance: Significantly worse than ensemble tree models.

4. Logistic Regression

- Reasons: Linear decision boundary.
- Limitations: Dataset relationships are nonlinear.
- Performance: Lower ROC AUC and balanced accuracy compared with tree models.

Typical ranking on this dataset (historical benchmarking pattern)

1. Gradient Boosted Trees
2. Random Forest
3. SVM with RBF kernel
4. Logistic Regression
5. Single Decision Tree

Interpretation

The dataset contains nonlinear feature interactions and temporal feature correlations. Tree ensembles capture these structures automatically through hierarchical splits and feature interactions. Linear models cannot represent these relationships.

Result: boosted tree ensembles consistently dominate benchmark performance on dataset 4538.

Me: what research supports this?

AI: There is no single canonical paper stating “dataset 4538 is best solved by Random Forest / XGBoost”. The support comes from three converging research directions:

1. The dataset’s structure matches the regime where tree ensembles repeatedly dominate

Machine Learning benchmark studies consistently show that tree based ensembles outperform linear models on tabular multiclass data with nonlinear feature interactions.

A large 2025 benchmark comparing tabular methods across many OpenML datasets found:

- XGBoost
- LightGBM
- CatBoost
- Random Forest

consistently outperform:

- Logistic Regression
- standard MLPs
- shallow linear baselines

because tree methods naturally capture:

- nonlinear thresholds
- feature interactions

- heterogeneous distributions

This directly matches dataset 4538, which contains motion derived numerical features with nonlinear class boundaries.

Ye, H.-J. *et al.* (2025) *A Closer Look at Deep Learning Methods on Tabular Datasets, A closer look at deep learning methods on tabular datasets*. Available at:

<https://arxiv.org/html/2407.00956v4> (Accessed: 19 March 2026). Kohli, R. *et al.* (2024)

Towards quantifying the effect of datasets for benchmarking. Available at:

https://ml.informatik.uni-freiburg.de/wp-content/uploads/2024/04/61_towards_quantifying_the_effect.pdf (Accessed: 19 March 2026).

2. OpenML 4538 is a gesture phase segmentation dataset, and gesture data historically favors nonlinear classifiers

GesturePhaseSegmentationProcessed comes from gesture phase segmentation research using body motion features extracted frame by frame.

The original gesture segmentation literature showed that linear boundaries are insufficient, and nonlinear margin based or tree based classifiers perform better because gesture phases overlap in feature space.

Support Vector Machines were already shown to outperform simpler linear separation in early gesture segmentation studies, which implies that any stronger nonlinear ensemble usually gains further improvement. [Gesture unit segmentation using support vector machines: Segmenting gestures from rest positions | Request PDF](#)

3. Ensemble methods repeatedly dominate comparable gesture classification tasks

Across gesture recognition literature, boosted trees and forests repeatedly outperform single trees and logistic regression because they reduce variance and capture temporal feature interactions.

Recent gesture classification work reports:

- XGBoost near state of the art accuracy
- Random Forest highly competitive baseline
- Single Decision Tree weaker due to instability

Guo, J. (2024) *Gesture recognition and classification based on Newton Raphson optimised XGBoost algorithm*. Available at:

https://www.researchgate.net/publication/383136862_Gesture_recognition_and_classification_based_on_Newton_Raphson_optimised_XGBoost_algorithm (Accessed: 19 March 2026).

Gunasar, T. *et al.* (2022) *Decision Forest based EMG Signal Classification with low volume dataset augmented with random variance Gaussian Noise*, *arXiv.org*. Available at:

<https://arxiv.org/abs/2206.14947> (Accessed: 19 March 2026).

Why this matters specifically for your three models

If your comparison is:

- Logistic Regression
- Random Forest
- Decision Tree

then expected historical ranking is:

Random Forest>Decision Tree>Logistic Regression

Reason:

- Random Forest: Many trees average variance away.
- Decision Tree: Captures nonlinear splits but unstable.
- Logistic Regression: Only linear hyperplanes.

Strongest research backed statement you can write

Prior tabular benchmark research indicates that ensemble tree methods such as Random Forest and boosted trees generally outperform linear classifiers on datasets with nonlinear feature interactions. Because OpenML dataset 4538 contains gesture derived motion features with nonlinear class boundaries, Random Forest is theoretically expected to outperform Logistic Regression and a single Decision Tree.

Important caution

Historically expected best $\neq$ guaranteed best on your split.

Small differences occur depending on:

- train/test split
- scaling
- hyperparameters
- random seed

Your own measured balanced accuracy and ROC AUC remain the authoritative result.

Appendix I

This submission is my own independent work and all sources used in the submission are referenced. Generative AI was USED for this submission, as outlined by the relevant references. For each reference, I provide a full record of prompts given to the generative AI tool, and responses received below. I have also included references that acknowledge the usage of AI in the reference list of this coursework, based on the guidance provided at: [AI and Assessment: Digital Learning & Tools](#)

Self-evaluation

1. Did not contribute to this criterion
2. Willing but not very successful

3. Average
4. Above Average
5. Outstanding

Access the work submitted			Contribution (1-5 for each group member, including yourself). Replace {member x} with member's name. List members in the same order as at the beginning of the submitted notebook.			
Project Part	Weight	Mark (0-10)	{Henry Longe}	{Henry Akwarandu}	{Thaw Zin Lin Myat}	{Nsetu Tarimo}
I. Code	4	8	4	3	5	3
II. Report	6	6				